Student Name: Narges Kari
Student ID: 402110821

Deep Reinforcement Learning

HOMEWORK 4 — MODEL-BASED RL

1 Monte Carlo Tree Search

Implementation 1: MCTS Theory Questions [20 Points]

1. Failure of UCT under misleading early estimates and delayed rewards.
2. Effect of neural policy priors in PUCT-based MCTS.

Answer:

1. Standard UCT relies on the average expected return (Q). In environments where a path initially yields negative or misleading rewards but contains a massive delayed reward much deeper in the tree, UCT will heavily penalize that branch early on. Without prior knowledge, it gets trapped in local optima and requires an exponentially large number of simulations to discover the delayed reward.
2. PUCT integrates a neural network's prior probability ($P(s,a)$) into the exploration term. This prior provides heuristic guidance, directing the search towards strategically promising branches even if initial Q-values are poor or unknown. It effectively prunes the search space. As visit counts ($N(s,a)$) increase, the influence of the prior decays, allowing the true simulated environment dynamics (Q) to eventually dominate and correct any inaccuracies in the neural network's prediction.

Question 1: When Can MCTS Become Confidently Wrong? [20 Points]

Consider a root state s_0 with two possible actions, a_1 and a_2. The true optimal action is a_2, but early simulations make a_1 look better. MCTS uses

$$U(s_0,a) = Q(s_0,a) + c\sqrt{\frac{\ln N(s_0)}{N(s_0,a)+1}}.$$

Assume $Q(s_0,a_1) = 0.65$, $N(s_0,a_1) = n_1$, $Q(s_0,a_2) = 0.35$, and $N(s_0,a_2) = n_2$. Answer the following:

- ◇ Derive the condition under which MCTS selects a_2, i.e., $U(s_0,a_2) > U(s_0,a_1)$.

- ◇ Explain how the exploration constant c affects recovery from the early wrong estimate.
- ◇ Suppose the good outcome under a_2 is only visible after depth d. Explain why MCTS may fail with a limited simulation budget.
- ◇ In what situation could naive depth search outperform MCTS?

Answer:

- ◇ **Derivation of the Selection Condition:** To determine when action a_2 is selected over a_1, we evaluate the strict inequality $U(s_0, a_2) > U(s_0, a_1)$. Substituting $N(s_0) = n_1 + n_2$ along with the given empirical action values into the UCT formula yields:

$$Q(s_0, a_2) + c\sqrt{\frac{\ln(n_1 + n_2)}{n_2 + 1}} > Q(s_0, a_1) + c\sqrt{\frac{\ln(n_1 + n_2)}{n_1 + 1}}$$

$$0.35 + c\sqrt{\frac{\ln(n_1 + n_2)}{n_2 + 1}} > 0.65 + c\sqrt{\frac{\ln(n_1 + n_2)}{n_1 + 1}}$$

  Consequently, isolating the exploration bonus terms on the left-hand side establishes the final selection criterion:

$$c\left(\sqrt{\frac{\ln(n_1 + n_2)}{n_2 + 1}} - \sqrt{\frac{\ln(n_1 + n_2)}{n_1 + 1}}\right) > 0.30$$

- ◇ **Impact of the Exploration Constant c on Recovery:** The constant c balances the exploitation-exploration trade-off. Intuitively, an excessively small c yields a greedy policy that locks into the misleading estimate a_1, preventing recovery. Furthermore, a larger c magnifies the exploration bonus, accelerating recovery by reducing the total root simulations required to overcome the 0.30 reward deficit. Consequently, proper calibration of c is vital to ensure timely error correction without over-exploring suboptimal paths.
- ◇ **Failure Under Delayed Rewards and Limited Budget:** If the utility of a_2 is hidden until a deep horizon d, a constrained simulation budget causes a severe sample-efficiency bottleneck. Intuitively, a reward at depth d can only update $Q(s_0, a_2)$ via backpropagation if a simulation trace successfully reaches that depth. Furthermore, because early noisy signals falsely favor a_1, the tree policy skews the limited budget toward expanding the shallow subtree of a_1. Consequently, if the budget terminates before the exploration bonus forces the expansion of a_2 down to depth d, the value remains uncorrected, leading to a wrong decision.
- ◇ **Analysis of Scenarios Where Naive Depth Search Outperforms MCTS:** Naive depth search outperforms MCTS in deceptive environments characterized by sparse, deeply delayed rewards and misleading shallow signals. Intuitively, MCTS suffers from an inherent selection bias, organically constructing an asymmetric tree based on early noisy statistics that trap the policy in local maxima. Furthermore, unguided depth searches distribute the sampling budget uniformly across horizons without being biased by shallow value estimates. Consequently, when shallow rewards are deceptive, an unbiased depth-exploration strategy possesses a significantly higher probability of discovering the deep optimal reward.

Algorithm 1: UCT Selection Rule

Input: root state s_0, statistics $Q(s,a)$, $N(s,a)$, exploration constant c
for each action $a \in \mathcal{A}(s_0)$ **do**
    Compute

$$U(s_0,a) = Q(s_0,a) + c\sqrt{\frac{\ln N(s_0)}{N(s_0,a)+1}}$$

end for
Select

$$a^\star = \arg\max_a U(s_0,a)$$

Question 2: PUCT, Neural Priors, and Biased Exploration [10 Points]

In neural MCTS, the tree policy may use

$$a^* = \arg\max_a \left[Q(s,a) + c_{puct} P(s,a) \frac{\sqrt{N(s)}}{1+N(s,a)} \right]$$

where $P(s,a)$ is the prior probability predicted by a policy network. Suppose $P(s_0,a_1) = 0.80$, $P(s_0,a_2) = 0.15$, $P(s_0,a_3) = 0.05$. The true best action is a_3, but the policy network gives it the smallest prior. Answer the following:

- Explain why $P(s,a)$ is useful in large action spaces.
- Explain why a wrong prior can be harmful with a limited simulation budget.
- If all $Q(s_0,a)$ values are initially equal, compare the exploration bonus of a_1 and a_3.
- Suppose $Q(s_0,a_1) = 0.4$, $Q(s_0,a_3) = 0.2$, $N(s_0,a_1) = 50$, and $N(s_0,a_3) = 1$. Write the inequality that determines whether a_3 will be selected next.
- Propose one modification that reduces over-reliance on the policy prior.

Answer:

- **Utility of Policy Priors in Large Action Spaces:** In massive action spaces, uniform exploration suffers from an unsustainable branching factor. Incorporating the policy prior $P(s,a)$ dynamically scales down the exploration bonus of unpromising actions from the outset. Consequently, this concentrates the simulation budget almost exclusively on a high-probability subset, effectively pruning the tree horizontally and preventing a combinatorial explosion.
- **Detrimental Effects of Erroneous Priors Under Limited Budgets:** An incorrect prior heavily penalizes the optimal action by severely suppressing its exploration bonus. Intuitively, under a restricted simulation budget, the search terminates prematurely while still exploring paths erroneously favored by the network. Consequently, the tree policy completely misses high-reward trajectories because it lacks the computational budget required for visit counts

to counteract the initial negative bias.

- ◇ **Comparison of Initial Exploration Bonuses:** When all action-values are initially equal, visit counts are symmetric, making the fraction $\frac{\sqrt{N(s_0)}}{1+N(s_0,a)}$ identical across choices. Let this shared term be κ. The exploration bonuses for a_1 and a_3 simplify directly to $B_1 = c_{puct}(0.80)\kappa$ and $B_3 = c_{puct}(0.05)\kappa$. Dividing these expressions yields:

$$\frac{B_1}{B_3} = \frac{0.80}{0.05} = 16$$

Consequently, action a_1 receives a bonus exactly 16 times larger than a_3, strongly biasing early iterations toward the suboptimal path.

- ◇ **Derivation of the Selection Inequality for a_3:** To select a_3 over a_1, we evaluate the strict inequality $PUCT(s_0, a_3) > PUCT(s_0, a_1)$. Assuming a_2 is out of immediate competition but has been visited n_2 times, the total root visit count expands to $N(s_0) = n_1 + n_2 + n_3 = 50 + n_2 + 1 = n_2 + 51$. Substituting the parameters yields:

$$0.2 + c_{puct}(0.05)\frac{\sqrt{n_2+51}}{1+1} > 0.4 + c_{puct}(0.80)\frac{\sqrt{n_2+51}}{1+50}$$

Grouping the exploration components isolates the operational inequality:

$$c_{puct}\sqrt{n_2+51}\left(\frac{0.05}{2} - \frac{0.80}{51}\right) > 0.2 \implies c_{puct}\sqrt{n_2+51} > \frac{408}{19}$$

- ◇ **Proposed Modifications to Mitigate Prior Bias:** To prevent locking into incorrect neural heuristics, the prior distribution can be blended with a stochastic perturbation. Intuitively, injecting random noise into root prior probabilities allocates a baseline exploration floor, preventing any single path from being completely suppressed. Alternatively, adding an independent standard UCT term scaled by a small constant guarantees that as $N(s) \to \infty$, the search asymptotically recovers global optimality regardless of initial network bias.

Algorithm 2: PUCT Selection Rule

Input: state s, statistics $Q(s,a)$, $N(s,a)$, prior $P(s,a)$, constant c_{puct}
for each action $a \in \mathcal{A}(s)$ **do**
    Compute

$$\text{PUCT}(s,a) = Q(s,a) + c_{\text{puct}}\, P(s,a)\, \frac{\sqrt{N(s)}}{1+N(s,a)}$$

end for
Select

$$a^\star = \arg\max_a \text{PUCT}(s,a)$$

2 Cross-Entropy Method

Question 1: CEM as Trajectory Optimization [15 Points]

Assume that CEM samples candidate action sequences from $a^{(i)}_{0:H-1} \sim \mathcal{N}(\mu, \Sigma)$, rolls them out using the model, and evaluates $J(a^{(i)}_{0:H-1}) = \sum_{t=0}^{H-1} r(s^{(i)}_t, a^{(i)}_t)$. Answer the following:

- ◇ Explain how CEM uses elite samples to improve the sampling distribution.
- ◇ Why is CEM usually better than simple random shooting?
- ◇ Why does CEM not require gradients of the dynamics model or reward function?
- ◇ What information from one CEM iteration is carried to the next?

Answer:

- ◇ **Refitting the Sampling Distribution via Elite Samples:** CEM isolates a pre-determined fraction of top-performing candidate action sequences as elite samples. Intuitively, these trajectories represent regions yielding higher cumulative rewards. To shift the sampling density toward these domains, the distribution parameters are refit via Maximum Likelihood Estimation (MLE) over the elite set. Consequently, the mean μ_{k+1} and covariance Σ_{k+1} are updated directly using the empirical statistics of these sequences, ensuring focused future sampling.
- ◇ **Comparative Advantage over Random Shooting:** Unlike memoryless random shooting which samples uniformly, CEM is a structured, iterative search process. Intuitively, the algorithm updates its search distribution using trajectories empirically verified to be superior. Furthermore, maintaining a statistical summary of elite sequences drives progressive convergence toward optimal regions. Consequently, CEM concentrates its computational budget on refining promising paths rather than exploring irrelevant spaces.
- ◇ **Gradient-Free Characteristics of CEM:** CEM operates as a derivative-free black-box routine, remaining agnostic to the analytical form of the dynamics and rewards. Intuitively, updates rely solely on scalar objective evaluations $J(a^{(i)}_{0:H-1})$, meaning only function values matter rather than local slopes. Moreover, avoiding backpropagation eliminates numerical instabilities like vanishing or exploding gradients. Consequently, the algorithm successfully optimizes complex, non-differentiable, or discontinuous landscapes.
- ◇ **Information Transferred Across Iterations:** Individual trajectory samples are discarded at the end of each iteration because CEM is entirely parameterized. Instead, historical knowledge is fully encapsulated within the updated mean μ and covariance Σ. Intuitively, μ tracks the high-reward center while Σ scales the search boundaries. Consequently, these statistical parameters serve as the sole informational bridge carrying the collective elite experience forward.

Question 2: Horizon Length and Dimensionality in CEM [10 Points]

Suppose the action dimension is d_a and the planning horizon is H. Therefore, the CEM search variable has dimension $D = Hd_a$. Answer the following:

- ◇ Explain why increasing H can improve planning quality.
- ◇ Explain why increasing H also makes CEM harder to optimize.

- If $d_a = 6$ and $H = 40$, compute the dimension of the trajectory optimization problem.
- Why can a limited number of samples cause CEM to miss good trajectories in high-dimensional action spaces?

Answer:

- **Impact of Horizon Length on Planning Quality:** Increasing H expands temporal foresight, allowing the planner to account for long-term downstream consequences and delayed rewards. Consequently, this eliminates myopic bias and prevents the optimization routine from settling for deceptive local optima that offer high immediate returns but lead to suboptimal future states.
- **Optimization Difficulties Induced by Extended Horizons:** A larger H expands the search dimension D, causing an exponential growth in the geometric volume of the continuous trajectory space. Moreover, propagating multi-step variations through non-linear dynamics creates an extensively rugged and non-convex reward landscape. Consequently, the sampling distribution becomes highly susceptible to premature convergence in poor local extrema.
- **Dimensionality Computation:** Given an action dimension $d_a = 6$ and a planning horizon $H = 40$, the total dimensionality D of the trajectory optimization problem is derived directly via the linear product formula $D = Hd_a$:

  $$D = 40 \times 6 = 240$$

  Consequently, the CEM algorithm must optimize a continuous 240-dimensional search space.
- **Effects of Sampling Sparsity in High-Dimensional Spaces:** In a 240-dimensional continuous space, a fixed sample budget N results in severe sampling sparsity across the vast geometric volume. Intuitively, the mathematical probability of any candidate sequence randomly hitting a narrow, high-reward region approaches zero. Consequently, because the elite fraction is selected from an unrepresentative, sparse sample set, CEM refits its parameters around suboptimal paths and misses the global optimum entirely.

Question 3: Open-Loop Weakness of CEM-Based Planning [10 Points]

CEM optimizes a full action sequence $(a_0, a_1, ..., a_{H-1})$ before executing the actions. Answer the following:

- Why is this considered open-loop trajectory optimization?
- What can go wrong if the real next state differs from the predicted next state?
- Explain how using CEM inside MPC can reduce this problem.
- Why does MPC not completely remove the difficulty of optimizing long action sequences with CEM?

Answer:

- ◇ **Open-Loop Nature of Pure CEM Planning:** The trajectory planning process is strictly open-loop because the entire sequence of actions is computed beforehand based solely on the initial state and a simulated transition model. Intuitively, during the execution phase, the agent executes these pre-determined actions sequentially without incorporating real-time state observations or environmental feedback. Consequently, the control inputs remain entirely uncoupled from any unexpected disturbances or state deviations that occur online.

- ◇ **Detrimental Effects of State Discrepancies:** If the true transition yields a state that differs from the model prediction, subsequent pre-planned actions are applied to unintended system states. Furthermore, because the system lacks a correction mechanism, these individual step deviations propagate and accumulate exponentially over the planning horizon. Consequently, this compounding error causes the actual trajectory to drift severely from the optimized path, driving the agent into suboptimal, unrewarding, or unsafe regions of the state space.

- ◇ **Error Mitigation via MPC Integration:** Embedding CEM within a Model Predictive Control framework introduces closed-loop receding-horizon feedback. Intuitively, instead of executing the full sequence, the agent applies only the first optimized action, observes the actual resulting state, and immediately re-optimizes the entire horizon from this new baseline. Furthermore, shifting the planning window forward dynamically discards past modeling errors before they can accumulate. Consequently, this continuous recalibration ensures robust tracking and significantly reduces vulnerability to model inaccuracies.

- ◇ **Persistence of Optimization Difficulties under MPC:** Although MPC successfully compensates for execution-time drift, it does not alleviate the fundamental mathematical complexity of the underlying search problem. At each discrete time step, CEM must still solve an independent optimization task over a high-dimensional space scaled by the horizon length:

$$D = Hd_a$$

  Moreover, the rugged, non-convex reward landscape and the risk of severe sampling sparsity remain identical. Consequently, the real-time computational burden persists, and the planner remains susceptible to premature convergence within individual planning cycles.

Algorithm 3: Cross-Entropy Method

1: ***Input:*** *current state s_t, model $\hat{f}$, reward r, horizon H, samples N, elite fraction ρ, CEM iterations K*

2: *Initialize sampling distribution over action sequences:*

$$a_{t:t+H-1} \sim \mathcal{N}(\mu_0, \Sigma_0)$$

3: ***for*** $k = 0, \ldots, K-1$ ***do***

4: *Sample N candidate action sequences:*

$$a^{(1)}_{t:t+H-1}, \ldots, a^{(N)}_{t:t+H-1} \sim \mathcal{N}(\mu_k, \Sigma_k)$$

for each candidate sequence $i = 1, \ldots, N$ ***do***

Roll out the sequence using the model:

$$\hat{s}^{(i)}_{t+j+1} = \hat{f}\left(\hat{s}^{(i)}_{t+j}, a^{(i)}_{t+j}\right)$$

Compute predicted return:

$$J^{(i)} = \sum_{j=0}^{H-1} \gamma^j r\left(\hat{s}^{(i)}_{t+j}, a^{(i)}_{t+j}\right)$$

end for

Select the top ρN elite sequences with the highest predicted returns.

Update μ_{k+1} and Σ_{k+1} using the mean and covariance of the elite sequences.

end for

Let $a^*_{t:t+H-1}$ be the best sequence found by CEM.

3 Model Predictive Control

Implementation 2: MPC Theory Questions [20 Points]

1. MPC as receding-horizon feedback.
2. Planning horizon and terminal value.
3. CEM-based optimization inside MPC.
4. Learned model errors and replanning.

Answer:

where?!

Question 1: MPC as Receding-Horizon Feedback [10 Points]

MPC optimizes an action sequence $a^*_{t:t+H-1} = (a^*_t, a^*_{t+1}, ..., a^*_{t+H-1})$ but executes only a^*_t. Answer the following:

- ◇ Explain why the optimization problem solved at time t is open-loop.
- ◇ Explain why the overall behavior of MPC is still feedback-based.
- ◇ Give a simple example where executing the whole planned sequence would fail, but replanning after each step would help.
- ◇ Explain why MPC is not the same as directly learning a closed-loop policy $\pi(a|s)$.

Answer:

- ◇ **Open-Loop Nature of the Local Optimization:** The optimization problem solved at time step t is structurally open-loop because the candidate action sequences are evaluated entirely within an internal forward model. Intuitively, the selection of the trajectory $a^*_{t:t+H-1}$ relies solely on simulated state predictions originating from the initial condition $\hat{s}_t = s_t$. Moreover, no external observations or online environmental measurements are injected to adjust the trajectory during the execution of the optimization algorithm itself. Consequently, the local optimization acts as an isolated, blind sequence generator over the horizon H.

- ◇ **Feedback-Based Global Behavior:** Globally, the overall behavior of MPC is feedback-based because the agent rejects the remaining pre-planned sequence and executes only the first optimal action a^*_t. Intuitively, after this execution, the agent observes the true, physical state s_{t+1} resulting from the environment. By continuously enforcing the initialization constraint $\hat{s}_{t+1} = s_{t+1}$ at the subsequent step, the next optimization cycle is forced to adapt to actual real-world transitions. Consequently, this receding-horizon mechanism embedding online state measurements effectively synthesizes a closed-loop feedback controller.

- ◇ **Illustrative Autonomous Driving Example:** Consider a vehicle driving on an unpaved, rough road where the internal dynamics model predicts that maintaining a fixed, straight steering angle will keep the vehicle centered. Under a purely open-loop execution of the full sequence, small unmodeled road bumps introduce microscopic lateral forces that accumulate over time, ultimately causing the vehicle to drift entirely off the road. Conversely, implementing MPC resolves this failure because replanning after each discrete step forces the planner to sense the actual lateral displacement caused by the bumps. Consequently, the next optimization cycle immediately commands a corrective steering adjustment, successfully maintaining the desired trajectory despite the rough terrain.

- ◇ **Distinction from an Explicit Closed-Loop Policy:** MPC differs fundamentally from directly learning an explicit closed-loop policy $\pi(a|s)$ because it does not represent a static, pre-computed mapping from states to actions. Intuitively, explicit model-free or parameterized policies amortize the action-selection process, computing inputs instantaneously during execution without any online future lookahead. In contrast, MPC expresses an implicit policy where the control input is generated dynamically by solving a computationally intensive numerical optimization problem online at every individual step. Consequently, MPC preserves explicit structural constraints and future foresight, whereas a learned reactive policy compresses this information into fixed function approximations.

Question 2: Planning Horizon and Terminal Value [10 Points]

The MPC objective uses a finite horizon H and may include a terminal value function:

$$\sum_{k=0}^{H-1} \gamma^k r(\hat{s}_{t+k}, a_{t+k}) + \gamma^H \hat{V}(\hat{s}_{t+H}).$$

Answer the following:

- ◇ What problem can happen if the horizon H is too short?

- ◇ What problem can happen if the horizon H is too long?
- ◇ Explain how the terminal value $\hat{V}(\hat{s}_{t+H})$ can help when H is short.
- ◇ Give one case where a wrong terminal value function can make MPC choose a bad action.

Answer:

- ◇ **Vulnerabilities of a Short Planning Horizon:** An excessively truncated horizon H introduces a severe myopic bias into the online optimization loop. Intuitively, if the planning window is shorter than the environment's inherent delay, the agent remains entirely blind to sparse downstream rewards or distant terminal hazards. Consequently, the local planner generates actions based purely on immediate gratification, trapping the policy in highly suboptimal local maxima or directing the agent into unavoidable physical failure modes situated just beyond the horizon boundary.
- ◇ **Limitations and Bottlenecks of a Long Planning Horizon:** Extending H excessively imposes a severe real-time computational burden, exponentially scaling the continuous trajectory search space for online optimizers like the Cross-Entropy Method. Furthermore, because any learned or estimated transition model $\hat{f}$ possesses non-zero epistemic errors, projecting state transitions deep into the future triggers an exponential accumulation of compounding errors. Consequently, far-horizon predictive rollouts become highly inaccurate, noise-dominated, and structurally misleading, causing the planner to optimize against simulated hallucinations rather than real-world physics.
- ◇ **Mitigation of Short Horizons via Terminal Value Functions:** The terminal value function $\hat{V}(\hat{s}_{t+H})$ serves as a compressed statistical proxy that encapsulates the expected cumulative tail rewards from the horizon boundary step $t+H$ spanning to infinity. Intuitively, incorporating this boundary term mathematically projects an infinite-horizon perspective back into a compact, finite planning window. Moreover, if $\hat{V}$ accurately approximates the true optimal value function V^*, solving a short-horizon MPC problem with $H = 1$ yields a policy equivalent to the global infinite-horizon optimum. Consequently, it restores long-term foresight without inflating online optimization costs.
- ◇ **Behavioral Failure Mode Induced by a Biased Terminal Value:** Consider an autonomous navigation task where a dangerous state (e.g., an unmapped dead-end or a cliff edge) resides at step $t + H$, but an inaccurate or poorly generalized neural network $\hat{V}$ erroneously maps that specific state to a massive positive value. Intuitively, the short-horizon planner will be deceived by this artificial numerical spike at the boundary. Furthermore, because the optimization routine strictly maximizes the total trajectory return, it will command steering actions that drive the vehicle directly into the hazard to capture the false terminal reward. Consequently, the wrong value function overrides local safety constraints, making the closed-loop MPC agent actively select a catastrophic control input.

Question 3: CEM-Based Optimization inside MPC [5 Points]

Assume that MPC uses the Cross-Entropy Method (CEM) to optimize the action sequence.CEM samples candidate sequences, evaluates them using the model, keeps the elite samples, and refits the sampling distribution. Answer the following:

- ◇ Explain why CEM is useful when the action space is continuous.
- ◇ If the action dimension is d_a, explain why the CEM search dimension is $D = Hd_a$.
- ◇ Why does increasing H make CEM harder, even though it gives the planner more foresight?
- ◇ Explain why executing only the first action of the best CEM sequence can make the method more robust than executing the full sequence.

Answer:

- ◇ **Utility of CEM in Continuous Action Spaces:** Continuous action spaces present an infinite branching factor that invalidates standard discrete tree-search techniques.Intuitively, CEM bypasses this restriction by utilizing a parameterized continuous probability distribution to sample smooth trajectory candidates.Consequently, the algorithm scales cleanly to multi-dimensional continuous vectors without requiring explicit discretization or analytical gradients.
- ◇ **Derivation of the CEM Search Dimension:** At each execution step, the planner must optimize an entirely integrated trajectory sequence spanning the full horizon H.Given that each individual action vector within this sequence possesses a dimension of d_a, the underlying optimization variable is formed by concatenating these temporal vectors.Consequently, the total search variable dimension simplifies linearly to the product $D = Hd_a$.
- ◇ **Optimization Hardness Induced by Extended Horizons:** Increasing H expands the parameter space D, which drastically amplifies both computational complexity and sampling sparsity across the non-convex reward landscape.Intuitively, as the search dimension expands, a fixed sample budget N becomes highly isolated, rendering the tracking of narrow reward regions statistically improbable.Furthermore, long-horizon forward rollouts compound multi-step predictive model errors.Consequently, this compounding drift distorts the objective function, causing CEM to suffer from severe computational overhead and premature convergence to suboptimal extrema.
- ◇ **Robustness of Receding-Horizon Execution via Closed-Loop Feedback:** Executing only the immediate control input a_t^* transforms an otherwise open-loop trajectory sequence into a closed-loop feedback mechanism.Intuitively, rather than blindly committing to a multi-step plan vulnerable to simulated errors, the agent receives true environmental feedback at the subsequent state transition.Moreover, this immediate influx of real state measurements allows the next CEM cycle to re-initialize from a corrected reality, effectively discarding future optimization hallucinations.Consequently, this continuous feedback loop prevents error accumulation and ensures robust tracking despite underlying model inaccuracies.